



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**VRLearnTobot Laboratorio
Virtual de programación y
robótica**



Presentado por Luis Ignacio de Luna Gómez
en Universidad de Burgos — 5 de julio de 2025
Tutor: Carlos López Nozal



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos López Nozal, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos..

Expone:

Que el alumno D. Luis Ignacio de Luna Gómez, con DNI 34809179J, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado VRLearn-Tobot Laboratorio Virtual de programación y robótica

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 5 de julio de 2025

Vº. Bº. del Tutor:

D. nombre tutor D. Carlos López Nozal

Resumen

El presente Trabajo Fin de Grado aborda el diseño e implementación de un prototipo de laboratorio virtual 3D para la enseñanza de los fundamentos de la programación y la robótica, con un enfoque en la accesibilidad y la motivación de estudiantes en etapas preuniversitarias. Desarrollado en el motor Unity con C#, el proyecto introduce un sistema de programación que combina una representación visual de la estructura del programa con la escritura de comandos textuales.

La principal contribución técnica es la creación de un Lenguaje de Dominio Específico (DSL) simple, junto con un parser que traduce el código fuente a una secuencia de instrucciones ejecutables, y un motor de simulación que interpreta dichas instrucciones para controlar un robot virtual en tiempo real. La arquitectura del sistema, inspirada en el patrón Modelo-Vista-Controlador (MVC), gestiona la interacción del usuario en un entorno de desafíos gamificados. El proyecto culmina en un Producto Mínimo Viable (MVP) que no solo demuestra la viabilidad técnica de la solución, sino que también funciona como un puente pedagógico, facilitando la transición de la programación por bloques a la codificación textual tradicional y alineándose con los Objetivos de Desarrollo Sostenible de democratizar la educación tecnológica.

Descriptores

Robótica Educativa, Unity, Programación, Lenguaje de Dominio Específico (DSL), Simulador 3D.

Abstract

This Final Degree Project addresses the design and implementation of a 3D virtual laboratory prototype for teaching the fundamentals of programming and robotics, with a focus on accessibility and motivation for pre-university students. Developed in the Unity engine with C#, the project introduces a programming system that combines a visual representation of the program's structure with the writing of textual commands.

The main technical contribution is the creation of a simple Domain-Specific Language (DSL), along with a parser that translates the source code into a sequence of executable instructions, and a simulation engine that interprets these instructions to control a virtual robot in real-time. The system's architecture, inspired by the Model-View-Controller (MVC) pattern, manages user interaction within a gamified challenge-based environment. The project culminates in a Minimum Viable Product (MVP) that not only demonstrates the technical feasibility of the solution but also serves as a pedagogical bridge, facilitating the transition from block-based programming to traditional textual coding, and aligning with the Sustainable Development Goals of democratizing technology education.

Keywords

Educational Robotics, Unity, Programming, Domain-Specific Language (DSL), 3D Simulator.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	5
2.1. Introducción	5
2.2. Objetivos funcionales	6
2.3. Objetivos no funcionales	7
3. Conceptos teóricos	9
3.1. Paradigmas de programación para la educación robótica . . .	9
3.2. Lenguajes de dominio específico (DSL) y su interpretación .	10
3.3. Simulación de robótica en entornos 3D con Unity	12
3.4. Aprendizaje basado en desafíos y objetivos de desarrollo sostenible	12
4. Técnicas y herramientas	15
4.1. Metodología de desarrollo: Agile con SCRUM	15
4.2. Herramientas core para el desarrollo de software	16
4.3. Herramientas adicionales	17
5. Aspectos relevantes del desarrollo del proyecto	19
5.1. Metodología de desarrollo: Iteraciones ágiles con SCRUM . .	19
5.2. Diseño e implementación de la arquitectura	20

5.3. Detalles clave de implementación	22
5.4. Punto de inflexión y diseño del sistema final	24
5.5. Retos técnicos abordados y estrategias de depuración	27
6. Trabajos relacionados	31
6.1. Plataformas de programación por bloques	31
6.2. Síntesis de la contribución del proyecto	34
7. Conclusiones y líneas de trabajo futuras	37
7.1. Conclusiones	37
7.2. Líneas de trabajo futuras	38
7.3. Conclusión final	43
Bibliografía	45

Índice de figuras

Índice de tablas

3.1. Resumen de la gramática y comandos del DSL implementado. .	11
6.1. Análisis comparativo de trabajos y herramientas relacionadas. .	32

1. Introducción

En la última década, el interés por la enseñanza de la programación y la robótica ha crecido de manera significativa, con especial énfasis en la educación preuniversitaria, dirigida a **niños y adolescentes**. Esto se debe a la creciente necesidad de dotar a las nuevas generaciones de habilidades fundamentales en pensamiento computacional, resolución de problemas y lógica algorítmica, consideradas esenciales para su desarrollo en el mundo digital actual [20]. La alfabetización digital y el pensamiento computacional son habilidades transversales cruciales que promueven la resolución creativa de problemas y preparan a los estudiantes para los desafíos de la sociedad moderna [10, 3].

Entre las metodologías más empleadas en la enseñanza de la programación, la **programación basada en bloques** ha demostrado ser una herramienta pedagógica eficaz. Este paradigma visual facilita la comprensión de la lógica de programación y el flujo de control sin la complejidad de la sintaxis textual, reduciendo la barrera de entrada para estudiantes novatos y promoviendo un aprendizaje más accesible y lúdico [21, 15]. Su efectividad se ha demostrado en diversos contextos educativos, como la incorporación de robótica en el currículo STEM a través de herramientas de bloques como Scratch [7]. Experiencias en la introducción a la programación en la universidad han mostrado que el uso de estos lenguajes ha mejorado significativamente las tasas de aprobación en asignaturas fundamentales de la informática [15].

Actualmente, dos de las plataformas más destacadas en este ámbito son **Scratch y Blockly**. Scratch, desarrollado por el MIT Media Lab, se ha consolidado como una de las herramientas educativas más utilizadas globalmente para la enseñanza de la programación, proporcionando un entorno altamente accesible y motivador [21]. Por otro lado, Blockly, desarrollado por Google,

ofrece una biblioteca de código abierto que permite integrar programación por bloques en diversas plataformas y puede convertir los bloques visuales a lenguajes de programación textuales como JavaScript y Python, ofreciendo así una mayor versatilidad en la generación de código y aplicaciones [15]. Aunque **Scratch utiliza una implementación personalizada basada en los principios de Blockly**, existen diferencias fundamentales entre ambos sistemas, especialmente en términos de flexibilidad y aplicación práctica [2]. La adopción y los avances en este campo se observan con gran desarrollo en diversas regiones de América Latina, donde la robótica educativa se ha posicionado como un medio eficaz para la enseñanza de habilidades STEM en docentes de primaria, evidenciando un compromiso temprano con estas metodologías [18].

El **aprendizaje basado en retos y la inclusión de elementos lúdicos** ha emergido como un enfoque clave en la enseñanza de la programación. Al introducir *desafíos con objetivos claros y un entorno interactivo*, se promueve una motivación intrínseca y un compromiso activo del estudiante en el proceso de aprendizaje, logrando así una experiencia más efectiva y significativa [2]. Estudios recientes confirman que los proyectos de robótica aumentan significativamente la motivación y el engagement de los estudiantes, facilitando un aprendizaje más profundo y aplicado [6]. Además, *los entornos interactivos con la programación (incluso con componentes de IA)* demuestran resultados positivos en el rendimiento académico y la motivación en estudiantes de primaria y secundaria [1]. Es en este cruce entre la rigurosa aplicación de los conocimientos técnicos y la creatividad del diseño, donde reside la oportunidad de desarrollar herramientas capaces de **vincular y emocionar a los estudiantes**, transformando el aprendizaje de la programación en una actividad lúdica y accesible que genere un impacto duradero.

No obstante, a pesar de estos avances metodológicos y didácticos, existen desafíos significativos para la enseñanza de la programación y robótica, especialmente en entornos con acceso limitado a recursos educativos y tecnológicos. La falta de laboratorios físicos y la dependencia de materiales y robots costosos restringen la accesibilidad de estas valiosas herramientas, afectando principalmente a estudiantes en zonas rurales o con restricciones económicas [20, 11]. La inviabilidad de replicar laboratorios con equipamiento sofisticado en contextos de bajo presupuesto y la dificultad de acceder a prácticas con componentes reales impulsan la búsqueda de alternativas flexibles y accesibles [20]. Esta problemática está directamente alineada con los **Objetivos de Desarrollo Sostenible (ODS)** de la Organización de las Naciones Unidas (ONU), en particular el **ODS 4 (Educación de**

calidad), que busca garantizar una educación inclusiva, equitativa y de calidad para niños y adolescentes [19]. Se enfoca en que nadie se quede atrás en el acceso a oportunidades educativas, y el **ODS 10 (Reducción de las desigualdades)**, que promueve la reducción de las brechas sociales y económicas entre regiones y poblaciones, buscando asegurar un acceso equitativo al conocimiento y a las oportunidades educativas [19].

Frente a esta problemática, los **entornos de simulación virtual y los laboratorios virtuales de robótica** han demostrado ser una solución excepcionalmente eficaz para *democratizar el acceso* a la enseñanza de la programación y robótica. Estas plataformas permiten a los estudiantes diseñar, construir y programar robots virtuales en un entorno digital, eliminando la necesidad de hardware físico costoso y ofreciendo un espacio de experimentación seguro y flexible. Permiten proporcionar una experiencia similar a la obtenida en un laboratorio de prácticas a través de sistemas de apoyo con gran realismo [16]. El uso de **juegos serios** (serious games) y simulaciones interactivas se ha consolidado como una metodología prometedora para la enseñanza de la programación, al integrar desafíos divertidos y elementos educativos que mejoran la retención del conocimiento e incrementan significativamente la motivación y la participación activa de los estudiantes [17].

El enfoque inicial de este proyecto consideraba la **replicación completa de un sistema de programación puramente visual basado en bloques, similar a Scratch, dentro de Unity**. Se diseñó una arquitectura robusta para la gestión, conexión y ejecución de bloques, sentando unas bases técnicas sólidas. Sin embargo, durante el desarrollo, se constató que la implementación de un motor de layout gráfico completamente dinámico para las vistas de los bloques presentaba una complejidad técnica que desviaba el foco del objetivo principal: la lógica de programación y la simulación del robot. Ante este reto de ingeniería, se tomó una decisión estratégica: pivotar hacia un Producto Mínimo Viable (MVP) que, aun manteniendo la filosofía de simplicidad y la representación visual del programa, se centrara en una interacción textual. Este reenfoque permitió concentrar los esfuerzos en el diseño de un lenguaje específico, su intérprete y la interacción con el entorno 3D.

Si bien el desarrollo de tecnologías en áreas como la simulación 3D o los lenguajes visuales es sólida, la *integración holística y accesible* de estos elementos en un *único sistema orientado a la robótica educativa específica de bajo coste* sigue presentando oportunidades significativas para el desarrollo y la implementación

de soluciones innovadoras. La creación de un puente pedagógico entre los sistemas puramente visuales y la programación textual tradicional, mediante un lenguaje de comandos simple y un entorno 3D inmediato, representa una oportunidad significativa. Este proyecto busca aportar valor en este nicho, ofreciendo una herramienta que facilita la transición hacia la codificación escrita junto con la programación por bloques.

En este contexto, el presente **Trabajo Fin de Grado** se orienta al desarrollo de un **prototipo de laboratorio virtual de programación y robótica**. La plataforma está diseñada con el motor de videojuegos **Unity** utilizando **C#**, con el objetivo de ofrecer una alternativa *accesible, interactiva y didáctica* para la metodología **STEAM**. La herramienta introduce un lenguaje de comandos textual y un intérprete propio, diseñados para ser intuitivos y controlar un robot virtual en un entorno 3D. El programa construido por el usuario se representa visualmente como una lista secuencial de instrucciones, manteniendo la claridad de un sistema de bloques pero fomentando la práctica con la escritura de código. El entorno de simulación está inspirado en la funcionalidad de kits como LEGO WeDo 2.0 y está optimizado para su uso en dispositivos Android tipo tablet.

Este proyecto plantea un desafío técnico centrado en el análisis sintáctico (parsing) y la interpretación de un lenguaje a medida. El sistema implementa un pipeline completo: desde la entrada de texto en una interfaz de usuario, su análisis y conversión a una estructura de datos intermedia (la lista de instrucciones), hasta la ejecución de dichas instrucciones en el motor 3D, gestionando el estado del robot y las interacciones con el entorno, como las colisiones.

La estructura de esta memoria detallará, en primer lugar, los objetivos específicos del prototipo. Posteriormente, se presentarán los conceptos teóricos que sustentan el desarrollo, incluyendo la arquitectura del intérprete de comandos y del motor de simulación. A continuación, se describirán las técnicas y herramientas empleadas. El capítulo central detallará los aspectos más relevantes del diseño e implementación del prototipo, seguido de una comparativa con trabajos relacionados. Finalmente, se expondrán las conclusiones obtenidas y las líneas de trabajo futuras para la evolución de este laboratorio virtual.

2. Objetivos del proyecto

2.1. Introducción

El presente TFG tiene como objetivo principal el desarrollo de un **prototipo de laboratorio virtual de programación y robótica enfocado en la enseñanza de la programación a través de un sistema de desafíos interactivos** que permita a los estudiantes aprender programación mediante bloques y código escrito. Desarrollado con el motor de videojuegos **Unity 6** y el lenguaje de programación **C#**, este laboratorio ofrece una plataforma que permite a los estudiantes aprender a programar tanto mediante **código escrito** como con **bloques visuales**.

La educación en robótica y programación ha demostrado ser una herramienta clave en el aprendizaje del pensamiento computacional. Sin embargo, uno de los principales desafíos en la enseñanza de la robótica es la dependencia de hardware físico, lo que limita el acceso a estudiantes en contextos de escasos recursos y en zonas menos favorecidas.

Para resolver este problema, el laboratorio virtual ofrece una solución dual. Por un lado, los estudiantes pueden programar el robot en un entorno 3D utilizando un sistema de bloques de colores, similar a Scratch, que permite arrastrar y conectar instrucciones de forma intuitiva. Por otro lado, y aquí reside la principal innovación, también pueden resolver los mismos desafíos escribiendo comandos directamente en un lenguaje de programación textual simple, diseñado específicamente para ser fácil de aprender. Ambas formas de programar controlan al mismo robot virtual, inspirado en kits como LEGO WeDo 2.0, permitiendo a los usuarios elegir el método que prefieran o transicionar de uno a otro.

Para guiar el desarrollo de este prototipo, se establecieron los siguientes objetivos específicos:

2.2. Objetivos funcionales

Estos objetivos están relacionados con las funcionalidades que el prototipo de laboratorio virtual debe cumplir para ofrecer una experiencia educativa inicial.

1. **Diseñar y desarrollar un sistema de programación para robótica educativa.**
 - Crear un entorno de programación que combine la programación visual por bloques con la programación textual.
 - Replicar el sistema de programación por bloques simulando a Scratch
 - Desarrollar un Lenguaje de Dominio Específico (DSL) [14] [12] textual para los comandos de bajo nivel del robot (mover, girar, repetir), y un **parser** para validar y procesar dichos comandos.
2. **Integrar el sistema de programación con un entorno de simulación 3D.**
 - Replicar la lógica fundamental de interacción de Scratch, permitiendo arrastrar, soltar y conectar bloques visuales en un área de trabajo.
 - Desarrollar un motor de ejecución (CSharpRunner) que interprete la secuencia de bloques visuales y la traduzca en acciones para el robot simulado.
 - Crear una librería de bloques básicos funcionales, incluyendo un bloque de evento para iniciar el programa, un bloque de control para repetir acciones y bloques de acción como mover pasos y girar
 - Diseñar la gramática de un Lenguaje de Dominio Específico (DSL) simple y orientado a la robótica (mover, girar, repetir, ...).
 - Desarrollar un parser que analice el código fuente del DSL, valide su sintaxis y lo transforme en una lista de instrucciones ejecutable.
 - Crear una interfaz de desarrollo (IDE) específica para este modo, donde el usuario escribe comandos y recibe feedback visual inmediato de su programa.

3. Implementar un prototipo de laboratorio virtual basado en desafíos.

- Crear un modelo 3D de un robot genérico inspirado en la estética de kits educativos como el robot Lego Wedo 2.0.
- Dotar al robot virtual de las funcionalidades de movimiento y rotación necesarias para completar los desafíos.
- Diseñar un sistema de gestión de desafíos que permita definir objetivos concretos, cargando dinámicamente las instrucciones y la configuración de la escena.

2.3. Objetivos no funcionales

Estos objetivos se enfocan en las cualidades transversales de la solución, como su accesibilidad, rendimiento y usabilidad.

1. Accesibilidad y escalabilidad del prototipo.

- Desarrollar una plataforma que pueda ejecutarse en dispositivos de hardware común y sin requerimientos técnicos elevados, con el objetivo de priorizar la portabilidad a plataformas Android (Tablets).
- Explorar estrategias de diseño y desarrollo que minimicen la dependencia de equipamiento costoso, haciendo la herramienta accesible a estudiantes en zonas con recursos limitados.

2. Investigación y análisis.

- Investigar y analizar plataformas existentes de programación y simulación robótica para comprender sus fortalezas y limitaciones.
- Definir qué aspectos de las soluciones existentes pueden ser mejorados o adaptados en la solución propuesta de este TFG, sentando las bases para futuras evoluciones del sistema.

3. Adquisición de conocimientos técnicos.

- Adquirir conocimientos y habilidades en el uso del motor Unity 6 y el lenguaje C# para la creación de entornos interactivos y simulaciones educativas.

- Aplicar buenas prácticas en el diseño de software y en la integración de sistemas de programación visual dentro de un motor de simulación, incluyendo la aplicación de patrones de diseño (MVC, Observador, Fábrica) y la gestión eficiente de estructuras de datos.

3. Conceptos teóricos

Este capítulo establece el marco conceptual sobre el que se sustenta el laboratorio virtual. Dado que el prototipo ofrece dos modalidades de programación distintas, este análisis abordará los fundamentos teóricos de ambos enfoques.

En primer lugar, se explorarán los principios de la **programación visual por bloques**, que sirvió como base para la arquitectura inicial y uno de los itinerarios de aprendizaje. A continuación, se detallarán los conceptos de los **Lenguajes de Dominio Específico (DSL)** y sus intérpretes, que constituyen el núcleo técnico del segundo itinerario. Finalmente, se expondrán las bases de la **arquitectura de simuladores 3D en Unity**, el entorno común donde ambos sistemas de programación convergen para controlar al robot. Este análisis teórico justifica las decisiones de diseño y la arquitectura final implementada.

3.1. Paradigmas de programación para la educación robótica

La programación visual por bloques, popularizada por plataformas como **Scratch** [21], es una herramienta pedagógica de probada eficacia para introducir el pensamiento computacional, al abstraer la complejidad de la sintaxis. Por otro lado, la programación textual es fundamental para el desarrollo de habilidades profesionales.

El prototipo desarrollado se sitúa en la intersección de ambos paradigmas, ofreciendo al usuario dos rutas de aprendizaje flexibles dentro del mismo laboratorio virtual.

La coexistencia de estos dos paradigmas dentro del prototipo no es casual, sino una decisión de diseño deliberada para crear un puente pedagógico. El estudiante puede comenzar con los bloques para asimilar conceptos abstractos de flujo y control, y posteriormente, aplicar ese conocimiento lógico en la modalidad textual, donde el desafío se centra en la sintaxis y la estructura del código escrito. De este modo, la herramienta acompaña al usuario en su curva de aprendizaje, desde los fundamentos visuales hasta la codificación práctica.

3.2. Lenguajes de dominio específico (DSL) y su interpretación

Una de las contribuciones técnicas centrales de este proyecto es el diseño e implementación de un lenguaje de dominio específico (DSL) [14] y su sistema de procesamiento. Un DSL [12] es un lenguaje informático diseñado para resolver problemas dentro de un dominio particular, en este caso, el control de un robot virtual básico.

El Proceso de un intérprete: De texto a acción

El procesamiento de un DSL sigue un pipeline similar al de un compilador, aunque simplificado para su ejecución directa. El sistema implementado en este TFG consta de dos fases principales:

- **Análisis (parsing):** El **parser** (`CommandParser`) es responsable del análisis léxico y sintáctico. Recibe el código fuente como una cadena de texto, lo divide en tokens (comandos y argumentos) y verifica si la secuencia cumple con la gramática definida para el DSL. Si la validación es exitosa, traduce el código a una estructura de datos intermedia, una lista de objetos `Instruction`, que funciona como un árbol de sintaxis abstracto (AST) simple y ejecutable.
- **Ejecución (interpretación):** El **intérprete** (`RobotController`) recibe la lista de instrucciones y la ejecuta de forma secuencial. Recorre la estructura de datos y, para cada instrucción, invoca la acción correspondiente en el entorno de simulación (ej. una corrutina de movimiento). Este modelo de ejecución directa es ideal para entornos interactivos, ya que proporciona feedback inmediato al usuario.

Gramática del DSL

Para definir el lenguaje, se ha establecido una gramática simple pero funcional, compuesta por un conjunto de comandos y reglas de sintaxis. Esta gramática define qué programas son válidos y cómo deben ser escritos por el usuario. La Tabla 3.1 resume los comandos principales.

Comando	Sintaxis	Descripción
<code>mover</code>	<code>mover <número> pasos</code>	Mueve el robot hacia adelante una cantidad de pasos especificada por <número>.
<code>girar</code>	<code>girar <dirección></code>	Rota el robot 90 grados sobre su eje. <dirección> debe ser izquierda o derecha.
<code>mover_durante</code>	<code>mover_durante <número> segundos</code>	Mueve el robot hacia adelante a velocidad predefinida durante un tiempo específico.
<code>repetir</code>	<code>repetir <N> veces</code> <code>repetir por_siempre</code>	Estructura de control para bucles. Requiere una instrucción de cierre <code>fin_repetir</code> .
<code>fin_repetir</code>	<code>fin_repetir</code>	Cierra un bloque de bucle <code>repetir</code> abierto.

Tabla 3.1: Resumen de la gramática y comandos del DSL implementado.

Además de la sintaxis de cada comando, el lenguaje sigue unas reglas generales:

- Los comandos no distinguen entre mayúsculas y minúsculas.
- Se ignora el espaciado extra entre los tokens de una misma línea.
- Las líneas en blanco o aquellas que comienzan con `//` se consideran comentarios y son ignoradas por el parser.

3.3. Simulación de robótica en entornos 3D con Unity

El laboratorio virtual se ha desarrollado sobre el motor **Unity**, elegido por su capacidad para crear simulaciones 3D interactivas y su robusto motor de físicas. Un **simulador 3D** en robótica educativa permite democratizar el acceso a esta disciplina, eliminando la dependencia de hardware costoso.

Arquitectura de la simulación

La simulación en este prototipo se ha estructurado siguiendo principios de separación de responsabilidades:

- **Entorno de simulación:** La escena 3D que contiene el escenario del desafío (muros, objetivos, etc.).
- **Agente robótico (RobotController):** Es el `GameObject` que representa al robot. Encapsula tanto su estado (posición, rotación) como su comportamiento (la lógica para moverse y girar). Interactúa con el entorno a través de su componente `Rigidbody` y detecta colisiones para reaccionar ante los límites del escenario.
- **Gestor de simulación (GameManager):** Actúa como el orquestador que conecta el sistema de programación con el de simulación. Recibe las instrucciones del parser y las delega al `RobotController`, además de gestionar el estado del desafío (inicio, fin, éxito, fracaso).

3.4. Aprendizaje basado en desafíos y objetivos de desarrollo sostenible

La eficacia de una herramienta educativa no solo reside en su tecnología, sino también en su enfoque pedagógico y su impacto social.

- **Aprendizaje basado en desafíos:** En lugar de un sandbox libre, el prototipo se estructura en torno a desafíos predefinidos (`ChallengeData`). Este enfoque proporciona al estudiante un objetivo claro, aumentando la motivación y contextualizando el aprendizaje de la programación como una herramienta para la resolución de problemas concretos.

- **Alineación con los ODS:** El proyecto contribuye directamente a los objetivos de desarrollo sostenible (ODS). Al ser una solución de bajo coste y accesible en dispositivos comunes, aborda el **ODS 4 (educación de calidad)** al democratizar el acceso a la formación STEM. Asimismo, al reducir la brecha digital entre estudiantes con y sin acceso a laboratorios físicos, apoya el **ODS 10 (reducción de las desigualdades)**.

4. Técnicas y herramientas

El presente capítulo tiene como objetivo describir las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Se detallarán los aspectos más destacados de cada elección, justificando el porqué de su utilización frente a posibles alternativas, y cómo estas herramientas han contribuido al logro de los objetivos planteados. Para mejorar la comprensión, las herramientas se han agrupado en subsecciones según su área de aplicación.

4.1. Metodología de desarrollo: Agile con SCRUM

El desarrollo del prototipo se abordó mediante el marco de trabajo **SCRUM**, estructurando el trabajo en *sprints* de corta duración. Esta metodología ágil fue fundamental para la adaptación continua, permitiendo una redefinición estratégica del alcance (pivote a un sistema de enseñanza de programación con dos itinerarios) tras identificar la complejidad del sistema de layout visual. La gestión del proyecto se realizó con **Zube** [23] para la planificación de tareas y el seguimiento del progreso en colaboración con el tutor, quien adoptó los roles de *product owner* y *scrum master*.

La adopción de SCRUM no solo ha facilitado la gestión del proyecto, sino que ha sido el pilar que ha permitido una toma de decisiones ágil ante desafíos técnicos, reorientando el esfuerzo hacia la creación de un MVP robusto y funcional.

4.2. Herramientas core para el desarrollo de software

Motor de videojuegos: Unity 6 y Lenguaje C#

El prototipo se desarrolló sobre **Unity 6** con **C#**, aprovechando su potente motor de renderizado 3D, su sistema de UI unificado (**Canvas**) y su capacidad de exportación multiplataforma, con un enfoque en dispositivos Android. El IDE principal fue **Visual Studio**.

Modelado 3D y creación de modelos de robot

El modelo del robot virtual se creó con **Blender**, tomando como referencia la estética de kits como LEGO WeDo 2.0.

Arquitectura de Programación: Sistema Dual

El núcleo del prototipo se sustenta en la implementación de dos sistemas de programación paralelos, ofreciendo al usuario dos itinerarios de aprendizaje distintos:

- **Base de programación visual (Adaptación de UBlockly):** Para el itinerario de programación por bloques, se partió de la librería de código abierto **UBlockly**. Se adaptó su modelo de datos (**BlockModel**, **ConnectionModel**) para implementar la lógica de los bloques estructurales (evento de inicio, bucles) y se refactorizó su arquitectura a un patrón MVC para una mejor organización. Esta base proporcionó un punto de partida sólido para el sistema visual.
- **Sistema de programación textual (DSL de desarrollo propio):** Para el itinerario textual, se diseñó e implementó desde cero un *Lenguaje de Dominio Específico (DSL)*. Esta fue la principal contribución técnica del proyecto e incluyó:
 - Un Parser (**CommandParser**) responsable del análisis léxico y sintáctico del código escrito.
 - Un Intérprete (integrado en el **RobotController**) que ejecuta la secuencia de instrucciones generada por el parser mediante corrutinas de **C#**.

Esta decisión permitió un control total sobre la semántica de las acciones del robot y eliminó la necesidad de un motor de layout visual complejo, enfocando el desarrollo en la lógica de programación y simulación.

Sistema de gestión de desafíos

La persistencia de datos en el prototipo se aborda de dos maneras complementarias:

- **Gestión de desafíos con Scriptable Objects:** La definición de los desafíos (título, descripción, etc.) se gestiona mediante **Scriptable Objects** (ChallengeData). Este enfoque de Unity permite desacoplar los datos del contenido de la lógica del juego, facilitando la creación y modificación de nuevos desafíos sin necesidad de cambiar el código. Un gestor (ChallengeLoader) se encarga de la persistencia de la selección del usuario entre escenas.

4.3. Herramientas adicionales

Para el control de versiones del código fuente se ha utilizado **Git** en conjunto con un repositorio en **GitHub**. Durante la implementación, se empleó **GitHub Copilot** como herramienta de asistencia a la codificación teniendo acceso a LLM's tales como Claude, Gemini, ChatGPT, etc. Finalmente, la presente memoria ha sido redactada en **LaTeX** utilizando el editor **TeXstudio**.

5. Aspectos relevantes del desarrollo del proyecto

Este capítulo tiene como objetivo recoger los aspectos más interesantes y no triviales del proceso de desarrollo del prototipo del laboratorio virtual de programación y robótica. Se detallarán las decisiones de diseño arquitectónico y de implementación clave, justificando los caminos adoptados y las estrategias empleadas para resolver los desafíos técnicos, desde la concepción inicial de un sistema de programación visual hasta la implementación final de la solución. Se busca que este apartado sirva como un resumen de la experiencia práctica del proyecto y una referencia útil para futuros desarrolladores.

5.1. Metodología de desarrollo: Iteraciones ágiles con SCRUM

El proceso de desarrollo de este prototipo ha seguido una metodología adaptativa, inspirada en los principios de **SCRUM**. Esta elección, frente a modelos más lineales, se justificó por la naturaleza experimental del proyecto y la necesidad de una *respuesta rápida* a los desafíos tecnológicos emergentes. La aplicación de este marco ágil permitió:

- **Entrega de valor incremental:** El proyecto se estructuró en ciclos de *sprints* de duración fija (ej., una a tres semanas), donde al final de cada iteración se obtenía un incremento funcional y probado del prototipo. Esto aseguró un progreso tangible y la validación temprana de funcionalidades críticas.

- **Flexibilidad y adaptación continua:** La revisión constante del progreso con el tutor (asumiendo roles de *product owner* y *scrum master*) permitió reevaluar prioridades, adaptar requisitos y refinar las soluciones técnicas en función de la experiencia práctica obtenida, así como gestionar la complejidad que surgió de la interacción entre UBlockly y Unity.
- **Transparencia y gestión del trabajo:** Herramientas como Zube [23] se emplearon para la planificación de *sprints*, el seguimiento de tareas con tableros *Kanban* y la asignación de *puntos de historia*. Esto facilitó la visualización del trabajo pendiente y realizado, mejorando la organización del proyecto siendo acompañado de comentarios, capturas de imágenes de la aplicación y videos donde se observaba la evolución del trabajo llevado a cabo para una correcta retroalimentación con el Tutor del TFG.

Este enfoque iterativo fue clave para la gestión eficaz de la complejidad técnica del proyecto y para alcanzar un *producto mínimo viable (MVP)* dentro del plazo establecido.

5.2. Diseño e implementación de la arquitectura

El punto de partida del proyecto fue el diseño de una arquitectura robusta para un sistema de programación puramente visual, tomando como referencia técnica la librería UBlockly [8]. A continuación, se detallan las decisiones clave de diseño tomadas en esta fase inicial, que sentaron las bases para todo el desarrollo posterior.

Adopción y mejora de UBlockly: Adaptación a MVC

Tras un análisis de la librería UBlockly, se identificó que, si bien proporcionaba un modelo de datos sólido, carecía de una separación estricta de responsabilidades entre la lógica y la representación visual. Como primera y fundamental aportación, se refactorizó su estructura para adaptarla al patrón arquitectónico Modelo-Vista-Controlador (MVC). Esta decisión fue crucial para gestionar la complejidad y sentar las bases de un sistema mantenible y escalable.

- **Modelo:** Las clases centrales del Modelo (`WorkspaceModel`, `BlockModel`, `ConnectionModel`, `InputModel`, `FieldModel`) encapsulan el estado lógico y las reglas de negocio del sistema de bloques. Son independientes de cualquier representación visual y se enfocan en la *abstracción de la programación visual* del usuario.
- **Vista (Representación visual e interacción UI):** Componentes Unity UI como `UnityEngine.UI.Image`, `TMPPro.TextMeshProUGUI`, `UnityEngine.UI.Button` actúan como los elementos visuales, gestionados por una jerarquía de clases `View` (`BaseView`, `BlockView`, `ConnectionView`, `InputView`, `FieldView`, `LineGroupView`). Estas vistas son las únicas que conocen la interfaz de Unity y se encargan de representar el Modelo.
- **Controlador (Orquestación e interpretación de interacciones):** Capas de lógica como `WorkspaceController`, `BlockController`, `TollboxController`, median entre el *modelo* y la *vista*. Interpretan las acciones del usuario, validan operaciones y orquestan las actualizaciones del Modelo y la Vista sin que estas capas se conozcan directamente entre sí.

La comunicación entre estas capas se gestiona primordialmente mediante el **patrón Observador** (`Observable<TArgs>` e `IObserver<TArgs>`). Los modelos (`BlockModel`, `ConnectionModel`, `FieldModel`) actúan como sujetos observados, notificando de cualquier cambio relevante a sus vistas o controladores suscritos. Esto asegura una *sincronización eficiente* de la interfaz sin acoplamientos rígidos entre los elementos.

Sistema de programación visual por bloques: Componentes clave

La decisión de basar la implementación del motor de bloques en el repositorio de **UBlockly** [8], una adaptación de Blockly en C# para Unity, fue una *elección estratégica* frente al desarrollo desde cero. Este enfoque se justificó por la necesidad de construir un prototipo funcional robusto en un tiempo limitado, ya que UBlockly proporcionó una base sólida para:

- **Motor lógico de bloques:** Sus clases core como `WorkspaceModel` (gestor principal de la base de datos de bloques), `BlockModel` (estructura lógica de un bloque) y `ConnectionModel` (sistema de conexiones entre bloques) ofrecieron una base abstracta y funcional ya testeada. Se adoptó este diseño tal cual, centrándose la adaptación en cómo estos modelos interactúan con la nueva capa de vistas en Unity.

- **Motor de ejecución asíncrono:** UBlockly ya incluye un robusto sistema de interpretación directa basado en `Cmdtors` y corrutinas de C# (`CSharpRunner`). Esta arquitectura se ha adoptado directamente, adaptando los intérpretes (`MotionBlockInterpreter`) para interactuar con los objetos simulados del robot.

La adaptación principal ha consistido en *acoplar las capas de modelo y ejecución de UBlockly con la nueva capa de visualización de Unity UI*, así como *customizar la lógica de interacción* (drag & drop) para lograr una experiencia más cercana a Scratch. Se han creado prefabs de `GameObjects` de Unity con la estructura de componentes `View` que reflejan los `BlockModels` lógicos, permitiendo la creación modular y una apariencia flexible de los bloques.

5.3. Detalles clave de implementación

Representación visual y composición de bloques en Unity

Uno de los mayores desafíos en el desarrollo de interfaces de programación visual es la correcta representación y sincronización entre el modelo lógico del programa y su visualización interactiva. En este proyecto, la aproximación se basa en la creación de **prefabs** de `GameObjects` de Unity para cada tipo de bloque (`BlockView`), replicando la estética de Scratch.

- **Creación de prefabs estructurados:** Cada `BlockView` es un prefab con una jerarquía de componentes `UnityEngine.UI` que se corresponden directamente con las estructuras lógicas del `BlockModel` (ej., `InputView` para entradas, `FieldView` para campos). Esta composición predefinida de prefabs acelera la instanciación de bloques en tiempo de ejecución.
- **Layout manual y sincronización de `RectTransform`:** A diferencia de los sistemas de layout automático de Unity, el proyecto emplea un **sistema de layout manual** (`BaseView.UpdateLayout()`) para los bloques. Este sistema calcula dinámicamente la posición (XY) y el tamaño (`CalculateSize()`) de cada `BlockView` y sus subcomponentes (`InputView`, `FieldView`, `ConnectionView`). Es crucial porque los bloques de programación visual tienen formas complejas (muescas, pestañas, huecos para entradas) que no se ajustan de manera óptima a la-

youts grid o verticales/horizontales simples. Cada vez que el modelo lógico de un bloque cambia de posición (`BlockModel.XY`), o su contenido o tamaño (`BlockModel.FireUpdate()` para `UpdateStates.Inputs` o `UpdateStates.ConnectionModels`), el `BlockView` es notificado y marca su layout como sucio (`NotifyLayoutDirty`). Posteriormente, en un ciclo de Unity, se fuerza un recálculo del layout completo o parcial mediante `LayoutRebuilder.ForceRebuildLayoutImmediate()`.

- **Visualización de conexiones :** Las conexiones lógicas (`ConnectionModel`) tienen su reflejo visual en las `ConnectionViews`.

El Motor de ejecución de bloques: De la lógica a la acción del robot

El corazón operativo del prototipo es el motor de ejecución de bloques, responsable de interpretar la secuencia de instrucciones del usuario y traducirlas en acciones en el entorno de simulación 3D.

- **Intérpretes de bloques (Cmdtors):** Cada tipo de bloque funcional (ej. movimiento) tiene una clase intérprete concreta que hereda de `Cmdtor` (`ValueCmdtor`, `VoidCmdtor`, `EnumeratorCmdtor`). Por ejemplo, `MotionBlockInterpreter` se encarga de extraer los parámetros de mover N pasos de un `BlockModel`.
- **Orquestación con CSharpRunner:** La clase `CSharpRunner`, un `MonoBehaviour`, es el orquestador principal de la ejecución. Mantiene una `Stack<IEnumerator>` de `CmdEnumerators` para gestionar el flujo de control (secuencia lineal, bucles, condicionales). Esto permite una *ejecución paso a paso no bloqueante* del programa.
- **Concurrencia con corrutinas:** La característica distintiva de este sistema es el uso intensivo de las **corrutinas de C# (`IEnumerator`, `yield return`)**. Cuando un intérprete de bloque (`Cmdtor`) invoca una acción que toma tiempo (ej., una animación de movimiento del robot, un retardo), este se traduce en un `yield return` en C#. El `CSharpRunner` *pausa la ejecución de ese bloque* y la *reanuda automáticamente* cuando la corrutina de la acción externa ha finalizado. Esto es crucial para lograr *animaciones fluidas* del robot o *retrasos precisos* sin congelar la aplicación. Por ejemplo, `MotionBlockInterpreter` llama a `BlockObserver._MoveRobot()` (una corrutina), y el `CSharpRunner` espera a que esta corrutina termine antes de pasar al siguiente bloque.

- **Sincronización con la simulación:** La clase `BlockObserver` actúa como un puente entre el motor de ejecución de bloques y el `GameObject` del robot simulado (`RobotBehaviour`). `BlockObserver` recibe las órdenes de alto nivel (ej. mover 10 pasos) desde los `Cmdtors` y delega estas acciones a `RobotBehaviour` (el script que controla el movimiento y la animación física del robot), asegurando que la simulación refleje con precisión la lógica del programa. `BlockObserver` también gestiona el feedback visual en la interfaz de usuario, como los mensajes de estado y el resaltado del bloque en ejecución.

5.4. Punto de inflexión y diseño del sistema final

Tras implementar y evaluar la arquitectura inicial, se constató un problema en la creación de los prefabs a utilizar. Si es cierto que se había conseguido simular el espacio de trabajo de scratch, con sus distintos paneles, y la creación de bloques visuales específicos para eventos y movimiento, obteniendo además comportamientos para la unión de bloques y el arrastre de estos pero a la hora de desarrollar bloques de tipo C (control) iba a suponer un esfuerzo de desarrollo que superaba el alcance del proyecto. Con el objetivo de entregar un producto educativo funcional, se tomó la decisión estratégica de pivotar a un sistema que combinara una representación visual simplificada con un sistema de programación textual dirigido desarrollando para ello un DSL propio.

Componentes clave de la nueva arquitectura

La arquitectura del prototipo final, aunque inspirada en los principios de separación de responsabilidades, se simplificó para adaptarse al nuevo enfoque. La interacción entre la entrada del usuario y la simulación del robot está orquestada por un conjunto de gestores especializados:

- **GameManager:** Actúa como el controlador central de la escena de programación. Es responsable de recibir el programa parseado desde la interfaz de usuario y de iniciar la ejecución en el `RobotController`. Además, gestiona el estado general del desafío, como la finalización de la ejecución o la detección de eventos críticos (colisiones).
- **IDE_UIManager:** Este componente gestiona toda la interfaz de usuario de la escena de codificación. Captura las entradas del usuario (nuevos

comandos de texto, clics en los botones ejecutar o borrar) y las comunica al **GameManager**. También se encarga de actualizar la UI con el feedback del sistema, como los mensajes de error del parser o los mensajes de estado de la ejecución. Es la Vista-Controlador principal de la interacción del usuario.

- **RobotController:** Aunque principalmente es parte del modelo de simulación (define el estado y comportamiento del robot), también actúa como un controlador de bajo nivel para sus propias acciones. Contiene la lógica de ejecución de las instrucciones (Instruction) individuales en forma de corrutinas, manejando directamente la física (Rigidbody) y la cinemática del robot.
- **CommandParser:** Si bien no es un controlador en el sentido tradicional de MVC, esta clase estática implementa la lógica de negocio fundamental para la traducción del código fuente a una estructura de datos ejecutable. Es una pieza clave que desacopla la entrada de texto del motor de ejecución del robot.
- **ChallengeLoader:** Funciona como un controlador de datos entre escenas. Utilizando el patrón Singleton, se encarga de persistir la selección del desafío del usuario desde la escena del menú y de proporcionar esta información al **GameManager** al iniciar la escena de programación.

Esta estructura, aunque más directa que un sistema MVC de bloques completo, mantiene una clara separación de responsabilidades: la UI gestiona la entrada/salida, el GameManager orquesta el flujo, el Parser traduce la lógica, y el RobotController ejecuta las acciones físicas.

Diseño del sistema de desafíos y del DSL

La implementación final se basa en los siguientes componentes:

- **Gestión de desafíos con Scriptable Objects (ChallengeData):** Para estructurar el contenido educativo de forma modular y escalable, se utilizó el sistema de Scriptable Object de Unity. Cada ChallengeData define un desafío con su título, descripción e instrucciones, permitiendo una fácil creación de nuevo contenido sin modificar el código. Un gestor singleton (ChallengeLoader) asegura la persistencia de la selección del desafío entre la escena del menú y la de programación.

- **La interfaz de desarrollo (IDE - IDE_UIManager):** Se implementó una interfaz de usuario específica para la resolución de los desafíos. Esta vista principal gestiona:
 - La presentación de la información del ChallengeData activo.
 - Un campo de entrada (`singleCommandInput`) para que el usuario escriba los comandos del DSL.
 - La visualización del programa como una lista de bloques de texto instanciados desde un prefab (`commandBlockPrefab`), proporcionando un feedback visual inmediato de la secuencia de comandos.
 - La comunicación de las acciones del usuario (ejecutar, limpiar) al `GameManager`.
- **Implementación del intérprete (parser y ejecutor):** En el núcleo del sistema se encuentra un intérprete para un lenguaje de dominio específico (DSL) diseñado a medida:
 - **Análisis léxico y sintáctico (`CommandParser`):** Este parser, aunque simple, realiza las funciones básicas de un compilador. Recibe el código fuente como una cadena de texto, lo divide en líneas y tokens (palabras). Mediante una estructura switch-case, identifica los comandos (mover, girar, repetir, ...), valida sus argumentos y sintaxis, y en caso de éxito, traduce cada línea a un objeto `Instruction`. El conjunto de estas instrucciones forma una representación intermedia del programa, análoga a un árbol de sintaxis abstracto (AST) simple.
 - **Manejo de estructuras anidadas:** El parser utiliza una pila (Stack) para gestionar correctamente los bloques de bucle repetir ... fin_repetir, asegurando que las instrucciones se aniden jerárquicamente en la estructura de datos final.

Implementación de la simulación y ejecución del robot

El `GameManager` actúa como el orquestador principal en la escena de simulación, mientras que el `RobotController` encapsula la lógica y física del robot.

- **Motor de ejecución de instrucciones:** El `RobotController` recibe la `List<Instruction>` generada por el parser. Inicia una corrutina principal (`ProcessInstructionList`) que itera sobre la lista. Por cada

instrucción, invoca una sub-corrutina específica (e.g., `MoveForwardRoutine`). El uso de `yield return` en estas llamadas asegura que una instrucción no comience hasta que la anterior haya finalizado, garantizando una ejecución secuencial predecible.

- **Física y detección de colisiones:** El robot utiliza un componente `Rigidbody` para interactuar con el entorno. Para la detección de colisiones, se implementó una estrategia proactiva mediante `Rigidbody.SweepTest`. Antes de cada movimiento, se realiza un barrido en la dirección del desplazamiento para prever si habrá una colisión. Si se detecta un obstáculo, el movimiento se limita a la distancia hasta el punto de impacto y se notifica al `GameManager` para que detenga la ejecución. Esto proporciona un feedback inmediato y realista al usuario.

5.5. Retos técnicos abordados y estrategias de depuración

El desarrollo de un sistema de esta complejidad, integrando una librería de lógica de bloques con el entorno 3D y UI de Unity, ha presentado desafíos técnicos significativos.

- **Sincronización de modelos y vistas en MVC con corrutinas:** El reto de mantener el `BlockModel` lógico (con posiciones `Vector2` abstractas) perfectamente sincronizado con el `RectTransform` visual en el `Unity UI Canvas` ha requerido una depuración exhaustiva. Problemas de desajustes visuales o saltos en la posición de los bloques (especialmente al soltarlos tras un arrastre o al apilarse), así como errores de estado (`BlockView` o `ConnectionView` mostrando un estado inconsistente con el modelo), han sido frecuentes. La complejidad aumenta con la naturaleza flotante de las coordenadas y la ejecución asíncrona de las animaciones del robot, donde la interfaz debe reflejar cambios precisos en momentos concretos.
- **Bases de datos de conexiones en memoria (`ConnectionDB`):** Las `ConnectionDBs` de `UBlockly` utilizan estructuras de datos como cuadrantes o segmentos espaciales para optimizar la búsqueda de conexiones cercanas durante el arrastre. Asegurar que cada `ConnectionModel` estuviera correctamente registrado y desregistrado de estas bases de

datos (`ConnectionModel.Location` debía estar perfectamente actualizada, y las funciones `ConnectionModel.AddConnection()` y `ConnectionModel.RemoveConnection()` sin errores de referencia a objetos ya destruidos) ha sido un punto crítico. La coherencia de estas estructuras internas, que son abstractas (en memoria) y no se ven directamente en la UI, es vital para que el sistema de snap funcione.

■ **Gestión de la jerarquía de `GameObjects` y disposición (`Dispose/UnPlug`):**

La correcta re-parentalización de `GameObjects BlockView` al conectarse y desconectarse (particularmente la lógica de bumping) y la eliminación de recursos ha requerido un cuidado extremo. Un error aquí podía llevar a fugas de memoria, bloques invisibles, bloques duplicados o zombies en la jerarquía, o que el robot saliera lanzado por una lógica de corrutinas mal coordinada que no espera la finalización de las animaciones.

■ **Estrategias de depuración para sistemas complejos y asíncronos:** Dada la complejidad y la interacción entre múltiples componentes MVC, corrutinas y la interfaz de Unity, las herramientas de depuración han sido esenciales:

- **Logueo exhaustivo y contextualizado (`Logger`):** Se ha implementado y utilizado de forma intensiva la clase `Logger`, una utilidad global para la emisión de mensajes `Debug.Log`, `Debug.LogWarning` y `Debug.LogError`. *Decenas de puntos críticos en el código incluyen `Debug.Log`'s detallados (a menudo comentados para evitar el ruido en la ejecución normal) que rastrean el estado exacto de variables, la entrada y salida de funciones, el estado de las corrutinas, y la pertenencia de objetos a bases de datos.* Estos logs han sido el principal mecanismo para seguir el flujo de control, identificar desajustes y diagnosticar problemas de temporización.
- **Herramientas visuales de depuración en Unity:** El editor de Unity ha permitido la inspección en tiempo real de la jerarquía de `GameObjects`, los valores de los `RectTransforms`, el estado de los componentes `BlockView` y `ConnectionView`. Las herramientas de **Debugging de la ConnectionDB (`ExportConnectionDBsButton`)**, desarrolladas para el prototipo, han sido fundamentales para *visualizar y exportar a JSON* el estado de las bases de datos de conexiones en memoria, permitiendo un análisis detallado de su coherencia. Esto ha sido invaluable para la depuración de la lógica

de arrastre y snap, y sobre todo para entender como funcionaba esta.

La combinación de estas estrategias ha sido fundamental para navegar por la complejidad del desarrollo, asegurar la estabilidad del prototipo y justificar las decisiones de diseño a través de la evidencia empírica generada en el proceso de depuración.

6. Trabajos relacionados

Este capítulo presenta un análisis comparativo de las plataformas y herramientas más relevantes en el campo de la robótica educativa y la programación visual. El objetivo es contextualizar el prototipo desarrollado en este Trabajo Fin de Grado, identificando fortalezas, debilidades y oportunidades en el estado del arte. Esta evaluación ha sido fundamental para definir la contribución única del sistema híbrido propuesto.

6.1. Plataformas de programación por bloques

Para comprender el panorama actual, se han analizado diversas soluciones, agrupándolas por su enfoque principal: plataformas de programación visual, entornos de simulación y herramientas de desarrollo. La Tabla 6.1 resume este análisis.

Tabla 6.1: Análisis comparativo de trabajos y herramientas relacionadas.

Plataforma	Paradigma de prog.	Fortalezas clave	Limitaciones para el proyecto
Plataformas de programación visual de referencia			
Scratch [21]	Bloques visuales	■ Interfaz altamente intuitiva y lúdica, ideal para principiantes. ■ Gran comunidad y abundancia de recursos educativos. ■ Enfoque en la creatividad (animaciones, historias).	■ Ecosistema cerrado, no diseñado para ser extendido o integrado. ■ Sin capacidad nativa de exportación a código textual. ■ Integración compleja con entornos 3D personalizados.
Blockly [15]	Framework de bloques	■ Código abierto, modular y altamente personalizable. ■ Generación de código a múltiples lenguajes (JS, Python). ■ Base técnica para innumerables proyectos educativos y comerciales [22, 9].	■ Es una librería para desarrolladores, no una aplicación para el usuario final. ■ Requiere un esfuerzo significativo de desarrollo para la capa de interfaz de usuario.
Entornos de simulación de robótica educativa			
Simuladores genéricos en Unity [16]	Diverso (Bloques, Texto, APIs C#)	■ Aprovechan toda la potencia del motor Unity para renderizado 3D y físicas. ■ Máxima flexibilidad para crear escenarios y robots personalizados.	■ Generalmente son proyectos de investigación o de nicho. ■ Suelen carecer de una interfaz de programación pulida para usuarios novatos.

continúa en la página siguiente ...

... continúa desde la página anterior

Plataforma	Paradigma de prog.	Fortalezas clave	Limitaciones para el proyecto
CoSpaces Edu [4]	Híbrido (Bloques, Texto JS/Python)	■ Plataforma web muy accesible, sin instalaciones. ■ Herramientas de creación 3D y soporte para VR/AR.	■ La simulación física de robots es a menudo más abstracta que en simuladores dedicados.
VEXcode VR & CoderZ [5]	Híbrido (Bloques, Texto)	■ Simuladores de alta fidelidad para robots específicos (VEX, LEGO). ■ Alineados con competiciones y currículos STEM existentes.	■ Fuertemente ligados a ecosistemas de hardware y licencias específicas. ■ Poca o nula flexibilidad para personalizar el entorno o los robots.
Herramientas y frameworks para Unity			
UBlockly [8]	Framework de bloques en C#	■ Base de código abierto para implementar la lógica de bloques en Unity. ■ Proporciona un modelo de datos sólido y un motor de ejecución por corrutinas.	■ Proyecto no mantenido activamente. ■ La documentación es limitada, requiere ingeniería inversa. ■ No incluye un motor de layout visual dinámico.

continúa en la página siguiente ...

... continúa desde la página anterior

Plataforma	Paradigma de prog.	Fortalezas clave	Limitaciones para el proyecto
Blocks Engine 2 [13]	Asset Plug-and-play	■ Solución rápida y lista para usar. ■ Abstrae la mayor parte de la complejidad de la UI de bloques.	■ Actúa como una caja negra, con menor control sobre el motor de ejecución y la lógica interna. ■ No alineado con el objetivo de este TFG de profundizar en el diseño de un intérprete.

6.2. Síntesis de la contribución del proyecto

Tras el análisis de las herramientas existentes, la contribución fundamental de este Trabajo Fin de Grado se centra en los siguientes puntos:

- **Un puente pedagógico entre lo visual y lo textual:** A diferencia de las plataformas que son o 100 % visuales (Scratch) o que se centran en la programación textual tradicional, este proyecto propone una solución diferente. El usuario construye su programa mediante la adición secuencial de bloques de texto, manteniendo la claridad estructural de la programación por bloques, pero al mismo tiempo practica la sintaxis de un lenguaje de comandos. Esto lo posiciona como una herramienta ideal para la transición del aprendizaje, un paso intermedio que facilita el salto de Scratch a lenguajes como Python.
- **Diseño e implementación de un lenguaje de dominio específico (DSL) y su intérprete:** El núcleo de la aportación técnica no reside en la adaptación de una librería existente, sino en el diseño e implementación desde cero de un pequeño sistema de compilación:
 - Un **DSL** con una gramática simple y orientada a la robótica.
 - Un **parser** que actúa como analizador léxico y sintáctico.
 - Un **motor de ejecución** que interpreta la estructura de datos generada por el parser.

Este ciclo completo de código fuente -> análisis -> ejecución es una demostración práctica de conceptos fundamentales de la ingeniería informática.

- **Integración completa en un entorno de simulación 3D accesible:** El proyecto integra esta lógica de programación en un entorno 3D interactivo construido en Unity, con un enfoque en la accesibilidad para plataformas móviles (tablets) y sin la dependencia de hardware costoso. Se logra así un laboratorio virtual completo y autónomo, alineado con los objetivos de desarrollo sostenible de democratizar la educación tecnológica.

En síntesis, este TFG no se limita a replicar una solución existente, sino que aporta un prototipo con un enfoque pedagógico y técnico original, abordando el nicho de la transición a la programación textual mediante un entorno de desafíos gamificado y accesible.

7. Conclusiones y líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo, destacando los logros alcanzados y los aprendizajes obtenidos. Este capítulo final presenta una síntesis de los resultados del proyecto, tanto funcionales como técnicos. Además, se ofrece un informe crítico detallado que expone las limitaciones del prototipo actual y delinea una serie de líneas de trabajo futuras, que permiten trazar una hoja de ruta para la evolución y expansión del laboratorio virtual de programación y robótica.

7.1. Conclusiones

El presente Trabajo Fin de Grado ha culminado con el desarrollo de un prototipo funcional de un laboratorio virtual para la enseñanza de la programación y la robótica, implementado en el motor Unity 6 con C#.

El proyecto ha logrado con éxito el objetivo principal de crear un entorno de aprendizaje, que ofrece dos itinerarios de programación paralelos: uno visual por bloques y otro textual mediante un lenguaje de dominio específico (DSL). Esta solución dual ha demostrado ser una plataforma viable para la enseñanza, funcionando como un puente pedagógico que facilita la transición de la programación visual a la textual.

Los principales logros y aprendizajes del proyecto incluyen:

- **Implementación de una arquitectura MVC robusta y adaptable:** Partiendo de la librería UBlockly, se diseñó e implementó con

éxito una arquitectura Modelo-Vista-Controlador. Esta refactorización, ha permitido gestionar la complejidad y ha demostrado ser lo suficientemente flexible para soportar todo el sistema de programación visual por bloques.

- **Desarrollo de un sistema funcional de programación visual:** Se ha replicado la lógica de interacción de Scratch para un conjunto básico de bloques. El sistema permite arrastrar, conectar y ejecutar secuencias de bloques de evento (`event_whenflagclicked`) y de movimiento (`motion_movesteps`), gestionando la simulación del robot a través de corrutinas para una animación fluida.
- **Creación de un intérprete para un DSL propio:** Como solución a los desafíos técnicos del layout visual, se diseñó e implementó un sistema de programación textual completo. Esto incluye la definición de la gramática de un DSL, un *CommandParser* para el análisis léxico y sintáctico, y un motor de ejecución en el *RobotController* que interpreta las instrucciones generadas.
- **Validación de un flujo de simulación 3D unificado:** Se ha verificado que ambos sistemas de programación (visual y textual) son capaces de controlar al mismo robot virtual en el entorno 3D. El robot responde coherentemente a las instrucciones, y el sistema de detección de colisiones proporciona un feedback inmediato y funcional.
- **Establecimiento de un marco educativo basado en desafíos:** Mediante el uso de *ScriptableObjects*, se ha creado una estructura escalable para definir desafíos de aprendizaje, dotando a la herramienta de un propósito pedagógico claro y un flujo de usuario completo, desde la selección del reto hasta su resolución.
- **Aplicación exitosa de metodologías ágiles:** La gestión del proyecto con SCRUM no solo facilitó la organización y el seguimiento, sino que fue indispensable para tomar la decisión estratégica de pivotar el alcance, demostrando la importancia de la flexibilidad en la ingeniería de software real.

7.2. Líneas de trabajo futuras

El presente proyecto establece una base sólida sobre la que se pueden desarrollar mejoras y ampliaciones significativas para que el laboratorio

virtual evolucione hacia una solución educativa integral. A continuación, se presentan algunas de las posibles líneas de trabajo futuras:

Extensión y diversificación del sistema de bloques

Inicialmente, el prototipo implementa un conjunto limitado de bloques para demostrar su viabilidad. Para una solución completa, se plantea:

- **Ampliación de categorías de bloques:** Incorporar todas las categorías necesarias para dotar al robot de más acciones (ej. Sonido, Apariencia, Sensores).
- **Implementación de bloques avanzados:** Extender la funcionalidad para incluir estructuras de control más complejas como bucles, condicionales, funciones (procedimientos y bloques My Blocks), listas y operaciones con variables y operadores lógicos/matemáticos avanzados. Esto implicaría la adaptación de los intérpretes (`Cmddtors`) y la generación de prefabs visuales para cada nuevo tipo de bloque.
- **Expansión del modelo de eventos:** Desarrollar la capacidad para gestionar y responder a un rango más amplio de eventos, más allá de la bandera verde, permitiendo interacciones más complejas con el entorno virtual.
- **Sistema de depuración visual interactiva:** Crear herramientas visuales en la interfaz que permitan a los estudiantes seguir el flujo de ejecución del programa paso a paso, visualizar el estado de las variables y detectar errores en tiempo real, facilitando la comprensión y la resolución de problemas de lógica.
- **Integración de bloques con hardware simulado detallado:** Extender los bloques para representar el comportamiento detallado de motores (velocidad, dirección) y sensores (distancia, color, toque) del LEGO WeDo 2.0.

Mejora de la simulación 3D e interactividad

Actualmente, la simulación se centra en el movimiento básico del robot. Para mejorar la experiencia de aprendizaje y la inmersión, se pueden introducir mejoras como:

- **Entornos dinámicos e interactivos:** Diseñar escenarios 3D donde el robot pueda interactuar con obstáculos, objetos móviles o elementos activables (puertas, palancas), y responder a estímulos del entorno simulado (ej. zonas con distinto tipo de terreno que afecten el movimiento, etc.).
- **Simulación de física avanzada:** Integrar el motor de físicas de Unity (**Rigidbody**, **Collider**) de forma más compleja para dotar al robot y a los objetos del entorno de un comportamiento físico más realista (ej. colisiones, gravedad, rozamiento), lo que afectaría de manera auténtica la movilidad del robot y sus interacciones.
- **Soporte para múltiples modelos de robots:** Expandir el sistema para incluir y permitir la programación de distintos modelos de robots educativos (además del LEGO WeDo 2.0 básico), como LEGO SPIKE Essential, LEGO SPIKE Prime o LEGO MINDSTORMS EV3, cada uno con sus características y modelos 3D detallados.
- **Realidad virtual (VR) y aumentada (AR):** Explorar la portabilidad y adaptación del prototipo a plataformas de Realidad Virtual (VR) o Aumentada (AR), utilizando las capacidades nativas de Unity para ofrecer experiencias aún más inmersivas y atractivas para el aprendizaje.

Evaluación de la efectividad educativa y usabilidad

Una limitación inherente a la fase de prototipado es la ausencia de una validación exhaustiva con usuarios reales. Para futuras adaptaciones, es fundamental realizar:

- **Estudios de usabilidad y experiencia de usuario (UX):** Realizar pruebas formales con grupos de estudiantes de diversas edades y niveles de experiencia, empleando metodologías como el test A/B o encuestas de satisfacción, para evaluar la facilidad de uso de la interfaz, la intuición del sistema de bloques y la efectividad general de la plataforma.
- **Análisis del impacto en el aprendizaje:** Diseñar y llevar a cabo estudios longitudinales para medir cómo la plataforma mejora las habilidades de programación y el pensamiento computacional de los estudiantes, en comparación con métodos de enseñanza tradicionales o con otras herramientas existentes.

- **Recopilación de métricas de interacción y comportamiento:** Implementar herramientas de análisis y telemetría para recolectar datos anónimos sobre cómo los estudiantes interactúan con el sistema (tiempo dedicado, bloques utilizados, errores comunes, caminos de solución), lo cual permitiría identificar áreas de mejora y optimizar la experiencia de aprendizaje.

Integración con hardware físico

Aunque el prototipo actual se centra en la simulación virtual, el objetivo a largo plazo incluye la interacción con robots reales. Las posibles mejoras en esta línea abarcan:

- **Conexión en tiempo real con LEGO WeDo 2.0 (Física):** Desarrollar un módulo de comunicación (mediante Bluetooth Low Energy) que permita la ejecución directa del código programado con bloques en el robot físico LEGO WeDo 2.0, creando un puente entre la programación virtual y la experiencia tangible.
- **Compatibilidad con otros kits de robótica:** Ampliar el sistema para permitir la programación y el control de otros dispositivos de robótica educativa, como Arduino o micro:bit, a través de interfaces de programación de bajo nivel (APIs o protocolos de comunicación específicos) para esos dispositivos.

Optimización del rendimiento, portabilidad y localización

Para maximizar la accesibilidad y el alcance del laboratorio virtual, es esencial abordar aspectos de optimización y portabilidad.

- **Optimización del rendimiento en Unity:** Llevar a cabo una fase de optimización exhaustiva para reducir el consumo de recursos de la aplicación (CPU, GPU, RAM), lo cual permitiría su ejecución más fluida en dispositivos de bajo rendimiento, como tablets básicas o computadoras más antiguas, maximizando la inclusión.
- **Portabilidad a versión web del entorno:** Desarrollar una versión ejecutable del laboratorio virtual directamente en navegadores web (utilizando tecnologías como Unity WebGL), eliminando la necesidad de instalación y facilitando un acceso instantáneo desde cualquier dispositivo con conexión a internet.

- **Localización del Sistema:** Traducir la interfaz de usuario, las descripciones de los bloques y los tutoriales a múltiples idiomas. Esto expandiría el alcance educativo de la plataforma a nivel global, eliminando barreras lingüísticas.

Reflexiones sobre el proceso de desarrollo y la adaptación

Una de las lecciones más significativas aprendidas durante este TFG ha sido la gestión de la complejidad técnica al adaptar repositorios de código abierto existentes. El objetivo inicial de replicar completamente el sistema visual de Scratch, tomando UBlockly como base, se reveló como un desafío de una magnitud considerablemente mayor a la anticipada.

El proceso de ingeniería inversa para comprender el funcionamiento interno de UBlockly, una librería con documentación limitada, requirió un gran esfuerzo de investigación y análisis. Lo que inicialmente parecía un problema sutil de implementación de layouts visuales se convirtió en un obstáculo significativo, especialmente en la recreación de bloques de control de flujo complejos (los denominados bloques de tipo C, como `if/else` o bucles), que requieren una manipulación dinámica de la geometría y la jerarquía de las vistas.

Lejos de ser un fracaso, este desafío representó un punto de inflexión crítico y formativo. Puso de manifiesto la importancia de la evaluación realista de los plazos y del alcance en un proyecto de ingeniería de software. La decisión de pivotar hacia un modelo diferente no fue un abandono del objetivo, sino una redefinición estratégica para garantizar la entrega de un producto funcional y coherente, centrando el esfuerzo en los aspectos donde se podía aportar mayor valor: el diseño de un lenguaje, su intérprete y la simulación interactiva.

Esta experiencia ha reforzado la conciencia sobre la dificultad de integrar sistemas complejos y ha subrayado la importancia de la modularidad y la arquitectura flexible, principios que guiaron tanto la refactorización inicial a MVC como el diseño final del prototipo. Los conocimientos adquiridos durante este proceso de adaptación y resolución de problemas constituyen uno de los aprendizajes más valiosos del proyecto.

7.3. Conclusión final

El desarrollo de este laboratorio virtual representa una trayectoria de ingeniería completa, desde la concepción de una idea ambiciosa hasta la entrega de un prototipo funcional y pragmático. La investigación inicial, centrada en la adaptación de UBlockly, demostró rápidamente la inmensa complejidad de replicar sistemas de programación visual como Scratch, especialmente en la generación dinámica de bloques de control de flujo. Este desafío, lejos de ser un fracaso, se convirtió en una valiosa lección sobre la gestión de la complejidad y la importancia de la adaptación estratégica.

El pivote hacia un enfoque diferente ha culminado en un prototipo que valida un modelo pedagógico original y técnicamente sólido. La aplicación de los conocimientos del Grado en Ingeniería Informática ha sido fundamental, no solo para implementar el sistema final de intérprete de comandos, sino también para analizar la arquitectura inicial, identificar sus obstáculos y tomar decisiones de diseño informadas. El proyecto resultante es, por tanto, una prueba de las habilidades de ingeniería desarrolladas y un reflejo del compromiso con la creación de herramientas educativas accesibles, en línea con los objetivos de desarrollo sostenible.

Las líneas de trabajo futuras, que incluyen retomar la implementación completa del sistema visual, no se plantean como una tarea pendiente, sino como el siguiente paso lógico en un proyecto de mayor envergadura. Personalmente, este Trabajo Fin de Grado ha sido un aprendizaje profundo en arquitectura de software, desarrollo avanzado en Unity y resolución de problemas complejos. La intención de continuar esta línea de investigación, posiblemente durante un futuro máster en diseño de videojuegos, subraya la pasión y el conocimiento generados por este TFG, que considero un primer paso firme y exitoso.

Bibliografía

- [1] Fahad Alshahrani, Areej Aldhahri, Abeer Aldhahri, and Zahid Ali. Gamified learning of block-based programming with artificial intelligence concepts for k-12 students: an experimental study. *Smart Learning Environments*, 11(1):33, 2024. [En línea; consultado el 29-mayo-2025].
- [2] Jesús R. Campaña, Ana E. Marín, María Ros, Daniel Sánchez, Juan M. Medina, María Amparo Vila, María Dolores Ruiz, Manuel P. Cuéllar, and María J. Martín-Bautista. Metodologías activas y gamificación en las asignaturas de iniciación a la programación. In *Actas de las JENUI - Vol. 1 (2016)*, Almería, jul 2016. [En línea; consultado el 12-febrero-2025].
- [3] Code Week EU. The importance of early coding education. <https://codeweek.eu/blog/the-importance-of-early-coding-education/>, 2020. [En línea; consultado el 29-mayo-2025].
- [4] Codespaces. Delightex is a company working towards offering innovative technologies that contribute to transforming education. <https://www.delightex.com/>, 2025. [En línea; consultado el 30-mayo-2025].
- [5] Rafael Arnay Coromoto León Cristian Manuel Ángel Díaz, Eduardo Segredo. Simulador de robótica educativa para la promoción del pensamiento computacional. 2020. [En línea; consultado el 28-enero-2025].
- [6] María José L. Domínguez, Juan R. Arévalo, David M. Pérez, Ignacio L. Durán, José P. D. López, and Manuel J. Gil. Motivation and engagement of students: A case study of automatics and robotics projects. *Sensors*, 24(10):384850438, 2024. [En línea; consultado el 29-mayo-2025].

- [7] Brandon Fuchs. Incorporating robotic technologies into stem curricula through scratch robotics. Master's thesis, Concordia University, Saint Paul, 2021. [En línea; consultado el 29-mayo-2025].
- [8] Imagicbell. UBlockly - Introducción a la Programación con Bloques en Unity, 2021. [En línea; consultado el 20-febrero-2025].
- [9] Florin Cătălin Ionescu and Radu V. Vasiu. Developing a programming-learning platform using block-based approach and iot device for students. In *ICERD 2020: EAI International Conference on Electronic Communications, Internet of Things and Cloud Computing*, pages 102–108. Springer, 2020. [En línea; consultado el 29-mayo-2025].
- [10] David Klass. The benefits of teaching kids to code. <https://www.bc.edu/bc-web/sites/bc-magazine/fall-2023-issue/linden-lane/the-benefits-of-teaching-kids-to-code.html>, 2023. [En línea; consultado el 29-mayo-2025].
- [11] Lenin Lemus, Alberto Llorens, M^a. Belén Bollo, and José Manuel Gómez. Empleo de laboratorios virtuales en el espacio europeo de enseñanza. In *Actas de las JENUI - Vol. 1 (2006)*, pages 523 – 530, Almería, jul 2006.
- [12] Mariano Luzzza, Mario Berón, Germán Montejano, Pedro Rangel Henriques, and Maria J. Pereira. Diseño y construcción de lenguajes específicos del dominio. 15(2):67–81, 2010. [En línea; consultado el 2-julio-2025].
- [13] MeadowGames. Blocks engine 2. <https://meadowgames.com/>, 2025. [En línea; consultado el 30-mayo-2025].
- [14] Carlos Enrique Montenegro Marín, Juan Manual Cueva Lovelle, Óscar Sanjuan Martínez, and Paulo Alonso Gaona García. Desarrollo de un lenguaje de dominio específico para sistemas de gestión de aprendizaje y su herramienta de implementación “kiwidsm” mediante ingeniería dirigida por modelos. 15(2):67–81, 2010. [En línea; consultado el 2-julio-2025].
- [15] Roberto Muñoz, Thiago S. Barcelos, Rodolfo Villarroel, Marta Barriá, Carlos Becerra, Rene Noel, and Ismar Frango Silveira. Uso de scratch y lego mindstorms como apoyo a la docencia en fundamentos de programación. In *Actas de las XXI Jornadas de Enseñanza Universitaria de la Informática, JENUI 2015*, Andorra La Vella, jul 2015. [En línea; consultado el 12-febrero-2025].

- [16] Héctor Pérez, José L. García, and Marta Fernández. Simulador 3d de robótica distribuido en unity para enseñanza de programación e inteligencia artificial. *ResearchGate*, 2024. [En línea; consultado el 20-febrero-2025].
- [17] John Smith and Jane Doe. Geobotsvr: A robotics learning game for beginners with hands-on learning simulation. *arXiv*, 2024. [En línea; consultado el 20-febrero-2025].
- [18] Juan P. Torres, Miguel A. Castro, María F. Flores, José I. Villagran, Pamela F. Muñoz, and Claudia I. Contreras. A methodological approach to the teaching stem skills in latin america through educational robotics for school teachers. *Educ. Sci.*, 11(1):25, 2021. [En línea; consultado el 29-mayo-2025].
- [19] Naciones Unidas. Educación, objetivos de desarrollo sostenible, 2023. [En línea; consultado el 20-febrero-2025].
- [20] Maximiliano Paredes Velasco and J. Ángel Velázquez-Iturbide. Una asignatura para la formación del profesorado en programación mediante lenguajes basados en bloques. In *Actas de las JENUI - Vol. 7 (2022)*, pages 343–350, La Coruña, jul 2022. [En línea; consultado el 11-febrero-2025].
- [21] Maximiliano Paredes Velasco, Jesús Ángel Velázquez Iturbide, Sergio Caverio Díaz, and Daniel Palacios Alonso. Mejora de una asignatura para la formación del profesorado en programación basada en bloques. In *Actas de las JENUI - Vol. 8 (2023)*, pages 269–276, Granada, jul 2023. [En línea; consultado el 11-febrero-2025].
- [22] Lei Wang, Xiangang Zhou, Zhiyong Wang, Qingsong Zang, and Chunqing Tan. Towards a block-based programming language and its web-based environment to teach deep learning. *Frontiers in Education*, 6:657895, 2021. [En línea; consultado el 29-mayo-2025].
- [23] Zube Inc. Zube - project development managemet for agile development. <https://zube.io>, 2025.